

Demonstration Coverage and Extrapolation

The Claim

A model that learns from demonstrations fails on hard problems not because it lacks the capability, but because the demonstrations did not cover that difficulty. Adding one demonstration at the harder level restores performance. The model weights do not change. The question does not change. Only the examples change.

The Task

We use an ordering task: colored bars on a 10x10 grid, sorted by height. The input grid shows bars in a random arrangement. The correct output shows the same bars sorted left to right, shortest to tallest. Complexity is the number of bars (2 to 8).

A task set contains three demonstration pairs (input and output) and one query (input only). The model must predict the query output after reading the demonstrations.

Covered condition: one of the three demonstrations matches the query complexity. If the query has 8 bars, at least one demo also has 8 bars.

Uncovered condition: all three demonstrations have at most 3 bars. The query can still have 8 bars.

The query grid is identical in both conditions for the same seed. Only the demonstrations differ.

The Model

We trained a decoder-only transformer (6 layers, 256 dimensions, 8 attention heads, 5 million parameters) on 10,500 covered tasks across all complexities. With the right demonstrations present, the model learned to solve every difficulty level.

At inference, we do not retrain or update the model. We change only the demonstrations it reads as input context.

Results

Complexity	Covered EM	Uncovered EM	Gap
2	100%	96%	4 pp
3	100%	0%	100 pp
5	100%	0%	100 pp
8	100%	0%	100 pp

EM is exact match: the model’s output grid matches the correct output cell for cell. At complexity 8, covered demos produce 100% accuracy. Uncovered demos produce 0%. The gap is 100 percentage points.

At complexity 2, the gap is only 4 points. The uncovered demos (2 to 3 bars) already cover that range, so the model succeeds either way.

This is not a bias-variance problem. The model’s capacity and weights are fixed. It can solve complexity 8. Success depends on whether the input context contains a demonstration at the right difficulty.

Connection to BDH-CQ

BDH-CQ is a 150-million-parameter model built on Pathway’s Brain-inspired Dragon Hatchling architecture (arXiv:2608.09888). It stores demonstrations in a recurrent synaptic state instead of a growing key-value cache. It scored 29.5% pass@2 on ARC-AGI-1 at \$0.00070 per task.

BDH-CQ shows the same coverage cliff. Table 3 of the report:

Task	Uncovered	Covered	Source
Ordering, length 8	0/24	12/24 (pass@1)	arXiv:2608.09888, Table 3
Nesting, depth 5	15/24	16/24 (pass@1)	arXiv:2608.09888, Table 3

The ordering task shows a sharp cliff: 0 to 12 correct with one added demonstration. The nesting task shows a smaller gain: 15 to 16. Some operations depend on demonstration coverage more than others.

The ladder experiment from the same report shows a gradual falloff for ordering. Accuracy stays high through complexity 5, then falls: 29/36 at 6, 8/24 at 7, 1/24 at 8. Propagation and copying tasks show no falloff (48/48 at all distances).

System Comparison

System	Adaptation method	Weight updates at inference	Cost per task
BDH-CQ	Recurrent state absorbs demos	No	\$0.00070
HRM	Gradient optimization on demo pairs	Yes	\$1.48

System	Adaptation method	Weight updates at inference	Cost per task
TRM	Learned identity embedding per puzzle	Yes	\$1.76

BDH-CQ’s costs were reported by an independent audit by co-authors at Bielik and NYU (arXiv:2608.09888). HRM and TRM require backward passes at inference, which increases cost by a factor of 2,000.

Limitations

Our model is trained on a single synthetic task family. The coverage cliff is specific to ordering. Other operations (like propagation) do not show this cliff, even in BDH-CQ’s results.

Our demo includes a Hebbian memory visualization. This is a simplified illustration of the BDH update rule from the BDH Explainer Chapter 2 (Pathway, “From Attention to Synapses”). It omits low-rank compression, the positional operator, excitatory and inhibitory circuits, and ReLU gating. It is not the official BDH implementation.

All BDH-CQ numbers are developer-reported. No public model weights exist. We did not run BDH-CQ or reproduce their results.

Demonstration adaptation is session-scoped. The model does not learn across tasks or retain knowledge from one session to the next. Consolidating fast synaptic state into durable weights remains an open problem (Dragon Hatchling, arXiv:2509.26507, Conclusion).

Key References

1. BDH-CQ report, arXiv:2608.09888, August 2026
2. Dragon Hatchling, arXiv:2509.26507, September 2025
3. Coconut (latent reasoning baseline), arXiv:2412.06769, December 2024